

Python — La visualisation de données

Document 30 — Seaborn

Série « Visualisation » · graphiques statistiques élégants

Table des matières

- 1. Pourquoi Seaborn
- 2. Première image : un tracé statistique
 - 2.1 Le format « tidy » (données en colonnes nommées)
 - 2.2 Le lien avec Matplotlib : `.plot` renvoie un Axes
- 3. Un graphique complet
- 4. Les trois familles de tracés
- 5. Axes-level et figure-level
 - 5.1 Les fonctions axes-level (un Axes)
 - 5.2 Les fonctions figure-level (un FacetGrid)
- 6. Thèmes, styles et palettes
- 7. Corrélations : heatmap et pairplot
- 8. Sauvegarde et export
- Récapitulatif
- Questions de vérification

1. Pourquoi Seaborn

Seaborn est bâti sur Matplotlib, comme Pandas `.plot`. Mais là où Pandas vise la rapidité, Seaborn vise deux choses précises : les graphiques statistiques (distributions, corrélations, comparaisons de catégories) et un style soigné par défaut. On obtient en une ligne un graphique à la fois juste et présentable.

L'autre idée forte : Seaborn parle le langage des DataFrames. On lui passe un tableau via `data=` et l'on désigne les colonnes par leur nom (`x=`, `y=`, `hue=`). Pas besoin d'extraire les valeurs à la main : on raisonne en colonnes.

Atout	Ce que ça apporte
Statistique	Agrégations et estimations intégrées (moyenne, densité, régression).
Style soigné	Couleurs, grille et polices agréables sans réglage.
Pensé DataFrame	On désigne les colonnes par leur nom via <code>data=</code> .
Comparaisons faciles	<code>hue / col / row</code> éclatent un graphique par catégorie.

Pourquoi ça compte. Seaborn n'est pas un concurrent de Matplotlib : c'est une couche au-dessus, pour les graphiques statistiques. On l'utilise pour comprendre la donnée (distribution des rendements, corrélation entre actifs), puis on affine si besoin avec Matplotlib.

2. Première image : un tracé statistique

2.1 Le format « tidy » (données en colonnes nommées)

Seaborn attend des données « tidy » (ordonnées) : une ligne par observation, une colonne par variable. C'est le format naturel d'un DataFrame Pandas. On désigne ensuite les colonnes par leur nom.

```
import seaborn as sns
import pandas as pd

df = pd.read_csv("marche.csv")    # colonnes : date, actif, rendement

sns.lineplot(data=df, x="date", y="rendement", hue="actif")
hue="actif" trace une courbe de couleur différente par valeur de la colonne actif, et construit la
légende automatiquement. Une seule ligne suffit à comparer plusieurs séries.
```

2.2 Le lien avec Matplotlib : .plot renvoie un Axes

Comme Pandas, les fonctions simples de Seaborn renvoient un objet Axes Matplotlib. On le récupère pour titrer, nommer les axes et exporter — tout le document 10 reste valable.

```
import matplotlib.pyplot as plt

ax = sns.histplot(data=df, x="rendement", bins=30)
ax.set_title("Distribution des rendements")

fig = ax.get_figure()
fig.savefig("distribution.png", dpi=150)
```

Pourquoi ça compte. Le réflexe reste le même qu'au document 20 : `ax = sns.<fonction>(…)` rend un Axes. Seaborn produit un graphique statistique élégant, Matplotlib finit le travail (titre, export).

3. Un graphique complet

Beaucoup d'aspects sont déjà soignés par défaut : palette, grille, taille des points. On complète avec les paramètres de la fonction et, pour le reste, avec l'Axes retourné.

```
ax = sns.scatterplot(
    data=df,
    x="rendement_es", y="rendement_nq",
    hue="regime",          # couleur par categorie
    size="volume",         # taille des points par volume
    alpha=0.6,
)
ax.set_title("ES vs NQ par regime de marche")
ax.set_xlabel("Rendement ES")
ax.set_ylabel("Rendement NQ")
```

Paramètre	Rôle
<code>data=</code>	Le DataFrame source (format tidy).
<code>x= / y=</code>	Colonnes en abscisse / ordonnée, désignées par leur nom.
<code>hue=</code>	Colonne qui détermine la couleur (une catégorie).
<code>size= / style=</code>	Colonne qui module la taille / le style des marqueurs.
<code>palette=</code>	Jeu de couleurs appliqué au hue.

La sémantique avant l'esthétique. hue, size et style ne sont pas de simples options de décor : ils encodent une variable supplémentaire. Un graphique Seaborn peut ainsi montrer trois ou quatre dimensions à la fois, lisiblement.

4. Les trois familles de tracés

L'API de Seaborn s'organise en trois familles, selon la question posée à la donnée. Connaître la famille aide à trouver la bonne fonction.

Famille	Fonctions	Question posée
Relationnelle	scatterplot, lineplot	Comment deux variables varient ensemble ?
Distribution	histplot, kdeplot, ecdfplot	Comment une variable est-elle répartie ?
Catégorielle	boxplot, violinplot, barplot	Comment une variable diffère par catégorie ?

```
# distribution des rendements par actif
sns.kdeplot(data=df, x="rendement", hue="actif", fill=True)
```

```
# dispersion des rendements par regime
sns.boxplot(data=df, x="regime", y="rendement")
```

Pourquoi ça compte. La structure est constante d'une famille à l'autre : data=, x=, y=, hue=. Apprendre une fonction, c'est presque les connaître toutes. Le choix se ramène à la question : relation, distribution, ou comparaison par catégorie ?

5. Axes-level et figure-level

Chaque famille existe en deux versions, et c'est la distinction centrale de Seaborn. Les fonctions axes-level dessinent dans UN Axes ; les fonctions figure-level gèrent toute une figure et savent l'éclater en sous-graphiques.

5.1 Les fonctions axes-level (un Axes)

scatterplot, lineplot, histplot, boxplot... dessinent dans un seul Axes et le renvoient. Ce sont elles qu'on insère dans une figure Matplotlib existante via ax=.

```
fig, ax = plt.subplots(figsize=(8, 4))
sns.boxplot(data=df, x="regime", y="rendement", ax=ax)
ax.set_title("Rendements par regime")
```

5.2 Les fonctions figure-level (un FacetGrid)

relplot, displot, catplot (et Implot, pairplot) pilotent toute la figure. Avec col= ou row=, elles créent une grille de sous-graphiques — un par catégorie. Elles renvoient un FacetGrid, pas un Axes.

```
# un nuage par regime, cote a cote
g = sns.relplot(
    data=df,
```

```
x="rendement_es", y="rendement_nq",
col="regime",          # une colonne de la grille par regime
kind="scatter",
)
```

Niveau	Caractéristiques
Axes-level	Dessine dans un Axes, le renvoie ; accepte ax= ; s'insère dans une figure.
Figure-level	Gère toute la figure ; col=/row= pour les grilles ; renvoie un FacetGrid.

Pourquoi ça compte. La règle pratique : fonction axes-level (scatterplot...) pour un graphique unique à intégrer soi-même ; fonction figure-level (relplot...) quand on veut éclater en grille par catégorie avec col= / row=. C'est la confusion la plus fréquente avec Seaborn.

6. Thèmes, styles et palettes

Le style soigné de Seaborn se règle globalement, en une ligne, avant de tracer. Trois leviers : le thème d'ensemble, le style du fond, et la palette de couleurs.

```
import seaborn as sns

sns.set_theme(style="whitegrid", palette="deep")    # thème global

# ou plus finement :
sns.set_style("darkgrid")        # fond et grille
sns.set_palette("viridis")       # couleurs
```

Réglage	Effet
set_theme(...)	Applique d'un coup style, palette et échelle (recommandé).
set_style(...)	Fond et grille : whitegrid, darkgrid, white, ticks.
set_palette(...)	Couleurs par défaut des catégories.
palette= (par appel)	Palette pour ce seul graphique.
set_context(...)	Échelle des éléments : paper, notebook, talk, poster.

Catégoriel ou continu. Pour des catégories distinctes (actifs, régimes), une palette qualitative comme « deep » ; pour une grandeur ordonnée (volume, corrélation), une palette continue comme « viridis » ou « rocket ».

Pourquoi ça compte. `sns.set_theme()` en début de script suffit le plus souvent : tout le reste, y compris les graphiques Matplotlib et Pandas qui suivent, hérite du style. Le document 999 (cheat sheets) recense les palettes.

7. Corrélations : heatmap et pairplot

Deux graphiques résument à eux seuls les relations dans un panier d'actifs — un usage quotidien en finance.

La heatmap. Affiche une matrice (typiquement de corrélation) en damier coloré. On la calcule avec Pandas, on la dessine avec Seaborn.

```
corr = df[["ES", "NQ", "CL", "GC"]].corr() # matrice de corrélation

sns.heatmap(
    corr,
    annot=True, # afficher les valeurs
    cmap="coolwarm", # rouge/bleu, centre sur 0
    vmin=-1, vmax=1,
)
```

Le pairplot. Fonction figure-level qui croise toutes les colonnes deux à deux : nuages hors diagonale, distributions sur la diagonale. Idéal pour explorer un jeu de variables d'un coup.

```
sns.pairplot(df[["ES", "NQ", "CL"]], diag_kind="kde")
```

Pourquoi ça compte. `annot=True` et un `cmap` centré (`coolwarm`, avec `vmin=-1`, `vmax=1`) rendent une matrice de corrélation immédiatement lisible : le rouge pour les actifs corrélés, le bleu pour ceux qui s'opposent.

8. Sauvegarde et export

L'export dépend du niveau de la fonction utilisée — c'est la conséquence directe de la section 5.

Fonction axes-level. On remonte de l'Axes à la Figure, comme avec Pandas, puis `savefig`.

```
ax = sns.boxplot(data=df, x="regime", y="rendement")
fig = ax.get_figure()
fig.savefig("rendements.pdf")
```

Fonction figure-level. L'objet `FacetGrid` renvoyé possède sa propre méthode `savefig` — pas besoin de remonter à la Figure.

```
g = sns.relplot(data=df, x="es", y="nq", col="regime")
g.savefig("nuages.png", dpi=150) # le FacetGrid sait se sauver
```

Pourquoi ça compte. Une fonction axes-level se sauve via `ax.get_figure().savefig()` ; une fonction figure-level se sauve directement avec `g.savefig()`. Pour le reste — formats, dpi, sauvegarder avant `show()` — tout vient du document 10.

Récapitulatif

- 1. Le rôle.** Graphiques statistiques (distributions, corrélations, catégories) au style soigné, bâtis sur Matplotlib.
- 2. Données tidy.** On passe un `DataFrame` via `data=` et l'on désigne les colonnes par leur nom (`x=`, `y=`, `hue=`).
- 3. La sémantique.** `hue`, `size`, `style` encodent des variables : un graphique montre plusieurs dimensions, lisiblement.
- 4. Trois familles.** Relationnelle, distribution, catégorielle — selon la question posée à la donnée.
- 5. Deux niveaux.** Axes-level (un Axes, accepte `ax=`) vs figure-level (un `FacetGrid`, `col=`/`row=` pour les grilles).
- 6. Thèmes.** `sns.set_theme()` règle style et palette d'un coup, pour toute la suite.
- 7. Corrélations.** `heatmap` (matrice `.corr()`) et `pairplot` pour explorer un panier d'actifs.
- 8. Export.** Axes-level : `ax.get_figure().savefig()`. Figure-level : `g.savefig()`.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Qu'apporte Seaborn que Matplotlib seul ne donne pas facilement ?

Réponse. Des graphiques statistiques tout faits (distributions, corrélations, comparaisons de catégories) et un style soigné par défaut, en une ligne. Le tout en raisonnant directement sur les colonnes d'un DataFrame.

Q2. Quel format de données Seaborn attend-il, et comment désigne-t-on les variables ?

Réponse. Le format « tidy » : une ligne par observation, une colonne par variable (un DataFrame Pandas). On passe le tableau via `data=` et l'on nomme les colonnes avec `x=`, `y=`, `hue=`, etc.

Q3. À quoi sert le paramètre `hue=` ?

Réponse. Il associe une couleur à chaque valeur d'une colonne catégorielle (un actif, un régime de marché) et construit la légende automatiquement. Il encode une variable supplémentaire, pas seulement une décoration.

Q4. Quelle est la différence entre une fonction axes-level et figure-level ?

Réponse. Une fonction axes-level (`scatterplot`, `boxplot`...) dessine dans un seul Axes, le renvoie et accepte `ax=` : on l'intègre dans une figure existante. Une fonction figure-level (`relplot`, `displot`, `pairplot`...) gère toute la figure, sait l'écarter en grille avec `col=`/`row=`, et renvoie un `FacetGrid`.

Q5. Comment régler le style global de tous les graphiques d'un coup ?

Réponse. Avec `sns.set_theme(style=..., palette=...)` en début de script. Style et palette s'appliquent à toute la suite, y compris aux graphiques Matplotlib et Pandas.

Q6. Comment sauvegarde-t-on un graphique selon le niveau de la fonction ?

Réponse. Axes-level : on remonte à la Figure avec `fig = ax.get_figure()` puis `fig.savefig()`. Figure-level : l'objet `FacetGrid` renvoyé a sa propre méthode, `g.savefig()`, sans passer par la Figure.

Document suivant (40) : *Plotly — graphiques interactifs (zoom, survol, info-bulles) et tableaux de bord web ; on quitte la sortie statique pour l'exploration dynamique.*